

Developing Retrieval Augmented (RAG) based LLM Systems from PDFs: Experience

 **Ayman Asad Khan**
Tampere
ayman.khan@tuni.

 **Md Toufique**
Tampere
mdtoufique.hasan@tuni.

arXiv:2410.15944v1 [cs.SE] 21

 **Kai Kristian**
Tampere
kai-

 **Jussi**
Tampere
jussi.rasku@tuni.

 **Pekka Abrahamsson**
Tampere
pekka.abrahamsson@tuni.

Abstract. This paper presents an experience report on the development of Retrieval Augmented Generation (RAG) systems using documents as the primary data source. The RAG architecture combines the generative capabilities of Large Language Models (LLMs) with the precision of information retrieval. This approach has the potential to enhance how we interact with and augment both structured and unstructured knowledge in generative models to enhance the accuracy and contextual quality of responses. The paper details the end-to-end workflow from data preprocessing to indexing and generation, highlighting technical challenges and practical solutions. It aims to offer insights to researchers and practitioners developing RAG systems using two distinct *OpenAI's API GPT Series* and *Llama's open-source models*. The practical outcomes of this research lie in enhancing the reliability of generative AI in various sectors where domain specific knowledge and real time information retrieval is important. The Python code used in this work is available at:

Keywords: Retrieval Augmented Generation (RAG), Large Language Models (LLMs), Generative AI in Software Development, Transparent

1 Introduction

Large language models (LLMs) are at generating human like content, but base AI models can't keep up with the constantly evolving information in dynamic sectors. They rely on static training data, leading to outdated or incomplete results. Thus they often lack transparency and accuracy in high

decision RetrievalAugmented Generation (RAG) presents a solution to this RAG systems in information from data sourceslike databasesor grounding the generated content accurate and current data making it ideal for knowledge intensive

In this report, we document our experience as a step-by-step guide to RAG systems that integrates PDF documents as the primary knowledge We discuss the design choice, development of system, and evaluation of the providing insights into the technical challenges encountered and the practical lutions applied. We detail our experience using both proprietary tools and open-source alternatives (Llama) with data offering guidance choosing the right strategy. Our insights are designed to help practitioners researchers optimize RAG models for precision, accuracy and transparency best suites their use

2 Background

This section presents the theoretical background of this study. Traditional erative models, such as GPT, BERT, or T5 are trained on massive datasets have a fixed knowledge cut off based on their training They only generate **black box** answers based on what they **know**, and this is notable in fields where information changes rapidly and better ity and traceability of responses is such as lega analysis customer service, or technical

2.1 What is

The concept of Augmented Generation (RAG) models is built on tegrating two core components of NLP: Information Retrieval (IR) and Language Generation The RAG first introduced by et al.[5] combines dense retrieval methods with large scale generative models produce responses that are both contextually relevant and factually By explicitly retrieving relevant passages from a large corpus and this information in the generation RAG models enhance the grounding of their outputs from the up-to-date

A generic workflow of Retrieval Augmented Generation (RAG) system, casing how it fundamentally enhances the capabilities of Large Language (LLMs) by grounding their outputs in real- relevant information is trated in the Unlike static models which generate responses based on closed-world knowledge, the RAG process is structured into the following steps:

1. Data

The workflow begins with the acquisition of relevant, domain specific data from various external sources, such as PDFs, structured documents, text files. These documents represent raw data important for building a lored knowledge base that the system will query during the retrieval

enhancing the model's ability to

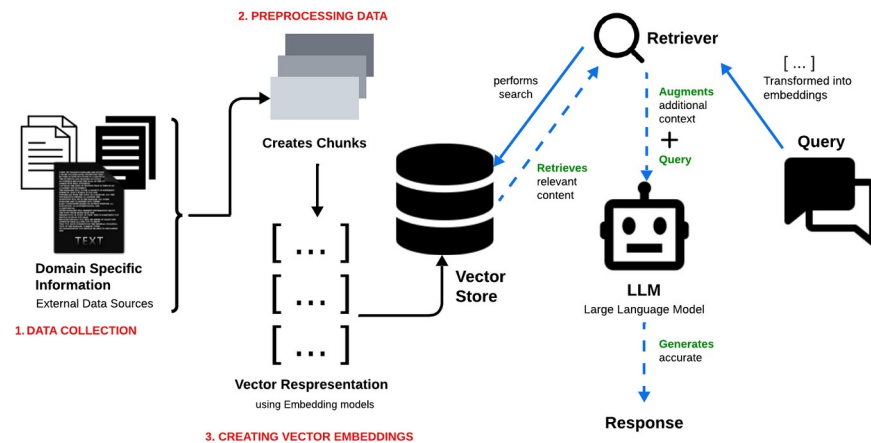


Fig. 1: Architecture of Retrieval Augmented Generation(RAG)

2. Data Preprocessing:

The collected data is then preprocessed to create manageable and functional chunks. Preprocessing involves cleaning the text (removing formatting), normalizing it, and segmenting it into smaller units, such as *tokens* (e.g., words or group of words), that can be easily indexed and later retrieved. This segmentation is necessary to ensure that the retrieval process is accurate and efficient.

3. Creating Vector

After data collection, the chunks of data are transformed into *vector representations* using embedding models like BERT or Sentence Transformers. These vector embeddings capture the semantic meaning of the text, allowing the system to perform similarity searches. The vector representations are stored in a **Vector Store**, an indexed database optimized for fast retrieval based on similarity.

4. Retrieval of Relevant Content:

When a **Query** is input into the system, it is first transformed into a *query embedding*, similar to the documents in the vector store. The system then performs a search within the vector store to identify and retrieve the most relevant chunks of information related to the query. This retrieval process ensures that the system uses the most pertinent and up-to-date information available in the vector store.

date information to respond to the

5. **Augmentation of Context:**

By merging two knowledge streams - the general knowledge deduced in the LLM and the flexible, domain-specific information augmented demand as an additional layer of context, aligns the Large Language (LLM) with both established and emerging

6. **Generation of Response by**

The *context-infused* consisting of the user query with the retrieved relevant content is provided to a Large Language (LLM) like GPT, T5 or Llama. The LLM then processes this augmented put to generate a coherent response not only fluent but factually

7. **Final**

By moving beyond the opaque outputs of traditional models, the final of RAG systems offer several advantages: they minimize the risk of hallucinations or outdated enhance interpretability by linking outputs to real-world enriched with relevant and responses

The RAG model framework introduces a paradigm shift in Generative AI creating **glass-box** models. It greatly enhanced the ability of generative to provide accurate information, especially in knowledge-intensive domains. integration has become the backbone of many advanced NLP applications, as chatbots, virtual assistants, and automated customer service systems.

2.2 **When to Use RAG: Considerations for**

Choosing between fine-tuning using Augmented Generation or base models can be a challenging decision for Each offers distinct advantages depending on the context and constraints of the case. This section aims to outline the scenarios in which each method is effective, providing a decision framework to guide practitioners in selecting appropriate

2.2.1 **Fine-Tuning: Domain Expertise and Customization**

Fine-tuning involves training an existing large language model (LLM) on a smaller, customized dataset to refine its knowledge for a particular domain or task. This excels in scenarios where accuracy, tone consistency, and deep understanding niche contexts are essential. For instance, fine-tuning has been shown to a model's performance in specialized content generation, such as technical ing, customer support, and internal knowledge

Advantages Fine-tuning embeds domain specific knowledge directly the model, reducing the dependency on external data sources. It is

effective when dealing with stable data or when the model needs to adhere to specific tone and

Drawbacks: Fine-tuning is computationally expensive and often substantial resources for training. Additionally, it risks overfitting if dataset is too narrow, making the model less

Use Case Examples:

- **Medical Diagnosis:** A fine-tuned model on medical datasets becomes specialized in understanding and generating medical advice based on terminologies and
- **Customer Support:** For a software company, fine-tuning on specific troubleshooting protocols ensures high-accuracy and consistent responses tailored to user

2.2.2 RAG: Dynamic Information and Large Knowledge Bases

Augmented Generation (RAG) combines LLMs with a retrieval mechanism allows the model to access external data sources in real-time, making it for scenarios requiring up-to-date or frequently changing information. RAG systems are valuable for handling vast knowledge where embedding the information directly into the model would be impractical or

Advantages: RAG is ideal for applications that require access to information, ensuring responses are grounded in real-time data and reducing hallucinations. It also provides transparency as the source of the retrieved information can be linked

Drawbacks: RAG requires complex infrastructure, including vector and effective retrieval pipelines, and can be resource-intensive during

Use Case Examples:

- **Investment Advisor Chatbot:** Using RAG, a chatbot can analyze the market trends and customer-specific portfolio data to offer personalized investment advice.
- **Document Analysis:** RAG can retrieve relevant case laws and statutes from a constantly updated legal database, making it suitable for applications where accuracy and up-to-date information are

2.2.3 When to Use Base Models

Using base models (without fine-tuning or RAG) is appropriate when the task requires broad knowledge, low-deployment, or rapid prototyping. Base models can handle simple use cases like generic customer support or basic question answering, where specialized or dynamic information is not

Advantages: No training is making it easy to and It is best for purpose tasks or when exploring applications without high upfront

Drawbacks: Limited performance on domain specific queries or tasks need high levels of

Table Decision Framework for Choosing Between Fine- RAG and Base

Factors	Fine-	RA	Base
Natur of the Tas	Highl specialize tasks domai spe- cific	Dynamic tasks ing tim infor mation	General tasks, typing broad cabilit
Data Require ments	Static or data tha rarel changes	Access to up-to- o externa larg knowledge	Does no requir specialized or up- date
Resource Con straints	High resources needed trainin	Higher inference and infrastruc tur complexit	Lo resourc de- mand, quic to deploy
Performance Goals	Maximizin pre cisio and adapt- abilit to specifi languag	Providin accu- rate context- response from namic	Optimizin speed and cost efficienc over

To conclude, the decision framework outlined in Table[1] offers a guide to selecting the most suitable method based on their project’s needsFine-Tuning is the best option for high-precision tasks stable RAG should be used when access to large-scale data necessaryand Base Models are well-suited for general-purpose use with resource

2.3 Understanding the Role of PDFs in

PDFs are paramount for RAG applications because they are widely used distributing high-value content like research lega documents,technica manuals, and financial reports, all of which contain dense, detailed essential for training RAG models. PDFs come in various forms, allowing to a wide range of data types—from scientific data and technical diagrams to gal terms and financial figures. This diversity makes PDFs an invaluable for extracting contextually relevant Additionallythe tent formatting of PDFs ensures accurate text extraction and context tion which is for generating precise PDFs also metadata (like keywordsand creation date) and annotations (such

highlights and comments) that provide extra context, helping RAG models organize sections and better understand document structure, ultimately improving retrieval and generation.

2.3.1 Challenges of Working with PDFs In RAG applications, text extraction from PDFs is essential for effective retrieval and generation. However, PDFs often feature complex layouts—such as multiple headers, footers, and embedded images—that complicate the extraction. These complexities challenge RAG systems which rely on structured text of high quality. Text extraction accuracy from PDFs decreases significantly in documents with intricate layouts, such as multi-column formats or documents with numerous figures and tables. This decline necessitates advanced techniques and machine learning models tailored to diverse document types.

Moreover, the lack of standardization in PDF creation, including different coding methods and embedded fonts, can result in inconsistent or garbled text, further complicating extraction and degrading RAG model performance. Additionally, many PDFs are scanned documents, especially in fields like law and academia, requiring Optical Character Recognition (OCR) to convert images to text. OCR can introduce errors, particularly with low-quality scans or handwritten text, leading to inaccuracies that are problematic in RAG systems where precise input is essential for generating relevant responses. PDFs may also contain non-textual elements like charts, tables, and images, disrupting the natural text flow required by most RAG systems. Handling these elements often requires specialized tools and preprocessing to ensure the extracted data is coherent and useful for RAG.

2.3.2 Key Considerations for PDF Processing in RAG

Development Processing PDFs for Augmented Generation applications requires careful handling to ensure high-quality text for effective retrieval and accurate generation. Below are key considerations specifically tailored for PDF processing in RAG.

1. Accurate Text

Since PDFs can have complex formatting, it is essential to use reliable tools and methods to convert the PDF content into usable text for further processing.

- **Appropriate Tools for Extraction:** There are tools and libraries for extracting text from PDFs for most popular programming languages (i.e. `pdfplumber` or `PyMuPDF (fitz)` for Python). These libraries handle common PDF structures and formats, preserving the text's layout and structure as much as possible.
- **Verify and Clean Extracted Text:** After extracting text, always verify it for completeness and accuracy. This step is crucial for catching extraction errors or artifacts from the PDF.

2. Effective Chunking for

PDF documents often contain large blocks of text, which can be challenging retrieval models to handle effectively. Chunking the text into smaller, ally coherent pieces can improve retrieval

- **Semantic** Instead of splitting text into small chunks, use semantic chunking based on logical divisions within the document, such as paragraphs or sections. This ensures that each chunk retains its context, which is helpful for both retrieval accuracy and understanding.
- **Dynamic Chunk Sizing:** Adjust the chunk size according to the type and the model's input. For example, scientific documents might be chunked by sections, while other types of documents could use paragraphs as the primary chunking unit.

3. Preprocessing and

Preprocessing the extracted text is key for removing noise that could affect performance both in retrieval and generative models. Proper cleaning of the text is consistent, relevant, and ready for further processing.

- **Remove Irrelevant Content:** Use regular expressions or NLP-based techniques to clean up non-relevant content like headers, footers, page numbers, and any repeating text that doesn't contribute to the document's main content.
- **Normalize** Standardize the text format by converting it to a consistent case, removing special characters, and trimming excessive whitespace. Normalization helps create consistent input for the retrieval model.

4. Utilizing PDF Metadata and

PDFs often contain metadata (such as the author, title, and creation date) and annotations that provide additional context, which can be valuable for tasks in RAG.

- **Extract Metadata:** You can use tools specific to programming like PyMuPDF or pdfminer.six for Python to extract embedded metadata. This metadata can be used as features in retrieval models, adding an additional layer of context for more precise search.
- **Utilize Annotations:** Extract and analyze annotations or comments in PDFs to understand important or highlighted sections. This can help prioritize content in the retrieval process.

5. Error Handling and

Reliability in processing PDFs is essential for maintaining the stability and consistency of RAG. Implementing proper error handling and logging helps manage unexpected issues and ensures smooth operation.

- **Implement Error Handling:** Use try-except blocks to manage errors during PDF processing. This ensures the application continues to run smoothly and logs any issues for later review.

- **Use Logging for Monitoring:** Implement logging to capture detailed information about the PDF processing steps, including successes, failures, any anomalies. This is important for debugging and optimizing the system over time.

By following these key considerations and best practices, we can effectively process PDFs for RAG, ensuring high-quality text retrieval, and generation. This approach ensures that your RAG models are efficient, and capable of delivering meaningful insights from complex PDF documents.

3 Study Design

This section presents the methodology for building a Retrieval Augmented Generation (RAG) system that integrates PDF documents as a primary source. This system combines the capabilities of information retrieval (IR) techniques with the generative strengths of Large Language Models to produce factually accurate and contextually relevant responses, grounded in domain-specific knowledge.

The goal is to design and implement a RAG system that addresses the limitations of traditional LLMs, which rely solely on static, pre-trained knowledge. By incorporating real-time retrieval from domain-specific PDFs, the system aims to deliver responses that are not only contextually appropriate but also up-to-date and factually accurate.

The system begins with the collection of relevant documents, including papers, legal documents, and technical manuals, forming a specialized knowledge base. Using tools and libraries, the text is extracted, cleaned, and formatted to remove irrelevant elements such as headers and footers. The cleaned text is then segmented into manageable chunks, ensuring efficient retrieval. These segments are converted into vector embeddings using transformer-based models like BERT or Sentence Transformers, which capture the semantic meaning of the text. The embeddings are stored in a vector database optimized for similarity-based search.

The RAG system architecture consists of two key components: a retriever, which converts user queries into vector embeddings to search the vector database, and a generator, which synthesizes the retrieved content into a coherent, natural language response. Two types of models are considered: OpenAI's GPT models, accessed through the Assistant API for ease of integration, and the open-source LLaMA model, which offers greater customization for domain-specific applications.

In developing the system, several challenges are addressed, such as handling complex PDF layouts (e.g., multi-column formats, embedded images) and maintaining high efficiency as the knowledge base grows. These challenges were highlighted during a preliminary evaluation process, where participants noted the difficulty of handling documents with irregular structures. Feedback from the evaluation also emphasized the need for improvements in text extraction and chunking to ensure coherent and accurate responses.

The design also incorporates the feedback from a diverse group of participants during a workshop session, which focused on the practical aspects of mentoring RAG systems. Their input highlighted the effectiveness of the real-time retrieval capabilities, particularly in knowledge-intensive domains, underscored the importance of refining the integration between retrieval and generation to enhance the transparency and reliability of the system's output. This design sets the foundation for a RAG system capable of addressing the needs of domains requiring precise, up-to-date information.

4 Results: Step-by-Step Guide to RAG

4.1 Setting Up the Environment

This section walks you through the steps required to set up a local environment for Retrieval Augmented Generation (RAG) on your machine. We cover the installation of Python, setting up a virtual environment, and configuring an IDE.

4.1.1 Installing Python If Python is not already installed on your system, follow the steps below:

1. Download and Install

- Navigate to the official Python website: <https://www.python.org/>
- Download the latest version of Python for your operating system (Windows, macOS, or Linux).
- During installation, ensure that you select the option *Add Python to PATH*. This is important to run Python from the terminal or command prompt.
- For Windows users, you can:
 - Click on *Customize*
 - Select *Add Python to environment*
 - Click *Now*

2. Verify the Installation

- Open the terminal (Command Prompt on Windows, Terminal on macOS/Linux).
- Run the following command to verify that Python is installed: `python --help`
- If Python is installed correctly, you should see output similar to the following:


```
python 3.x.x help
python 3.x.x --help
```

4.1.2 Setting Up an IDE After installing Python, the next step is to set up an Integrated Development Environment (IDE) to write and execute Python code. We recommend Visual Studio Code, however you are free to choose your own editor. Below are the setup instructions for VSCode.

1. Download and Install

- Visit the official VSCode website:
- Select your operating system (Windows, macOS or Linux) and the instructions for

2. Install the Python Extension in

- Open
- Click on the *Extensions* tab on the left-hand side (it looks like a book icon with four
- In the Extensions Marketplace, search for
- the Python extension by Microsoft. This extension allows VSCode support Python

4.1.3 Setting Up a Virtual Environment

you to install libraries and dependencies specific to your project without other projects on your

1. Open the Terminal in

- Press Ctrl + ` (or Cmd + ` on Mac) to open the terminal in
- Alternatively, navigate to *View -> Terminal* in the
- In the terminal, use the `mkdir` command to create a new folder for project. For example, to create a folder named `my-new-project`,
`mkdir my-new-project`
- Use the `cd` command to change directories and navigate to the directory where your project is located. For example,
`cd path/to/your/project/folder/my-new-project`

2. Create a Virtual Environment

- For Windows, run the following

```
python -m venv my_rag_env
my_rag_env\Scripts\activate
```
- For Mac/Linux, run the following

```
python3 -m venv my_rag_env
source my_rag_env/bin/activate
```

3. Configure VSCode to Use the Virtual Environment

- Open the Command Palette by pressing Ctrl + Shift + P (or Cmd + Shift + P on Mac)
- Type *Python: Select Interpreter* in the Command Palette
- Select your virtual environment, `my_rag_env`, from the list

With `virtualenv` you are ready to project specific dependencies and manage Python packages independently each. This setup allows you to create separate environment for the two approaches outlined in Sections [4.2.1](#). By isolating your dependencies you can ensure that the Assistant API- and Llama-based Retrieval-Augmented Generation (RAG) systems are developed and managed in their respective environments without conflicts or dependency. This practice also helps maintain more manageable development workflows for both models, ensuring that each approach optimally with its specific

4.2 Two Approaches to RAG: Proprietary and Open source

This section introduces a structured guide for developing Retrieval Generation (RAG) systems, focusing on two distinct approaches: *using Assistant API (GPT Series)* and an *open-source Large Language Model (LLM) Llama* and thus divided into two subsections [4.2.1](#). The objective is equip developers with the knowledge and practical steps necessary to RAG systems effectively, while highlighting common mistakes and best at each stage of the. Each subsection is designed to provide insights into setup, development, integration, customization and optimization generate well-grounded and aligned

In addition to the two primary approaches discussed in this guide there several alternative frameworks and methodologies for developing Retrieval mented Generation (RAG) systems. Each of these options such as Cohere, Jurassic-2, Google's PaLM, and Meta's OPT have their merits and trade-offs terms of deployment flexibility, cost, ease of use, and

We have selected **OpenAI's Assistant API (GPT Series)** and for this guide based on their wide proven and strengths in developing RAG. As highlighted in comparison *OpenAI's AP* provides a simple and developer-friendly black-allowing quick integration and deployment without the need for extensive management or infrastructure setup with high quality. In as an open-source Llama allows developers to have contro over model's architecture, training data, and fine-tuning process, allowing for customization to suit specific requirements such as demand flexibility and cost-. This combination makes these two options highly for diverse RAG system development

Table 2: Comparison of RAG Approaches: OpenAI vs.

Feature	OpenAI's Assistant (GPT)	Llam (Open-LLM)
Ease of Use	High Simple API calls no model management	Moderate Require setup and model management
Customizatio	Limited to prompt ing and few-shot	High Ful accessto model fine-tuning and
Cost	Pay-per-use pricing	Upfront infrastructure no API
Deployment Flexi-bilit	Cloud- depends on OpenAI's	Highly can be de- ployed locally or in any environmen
Performance	Excellent for a wide range general NLP	Excellentparticularly fine- fo specifi do- mains
Security and Data Privacy	Data processed on nA servers privac concerns may	Ful contro ove data and model;suitable for application
Support and Maintenanc	Strong documenta- tion, and updates from nA	Community- updates and support depend on munity
Scalabilit	Scalabl through cloud	Scalabl dependin on in frastructure
Contro Over Up- dates	Limited dependson Ope- nAI's release	Full control; users can when and how to update modify the

4.2.1 Using OpenAI's Assistant API : GPT Series While the Completion API is effective for simple text generation tasks, the Assistant is a superior choice for developing RAG. The Assistant API **multi-operations** (such as text, images, audio and video inputs) combining text generation with file, code, and calls. For a RAG this means an assistant can retrieve, generate vector embeddings, search for relevant content, augment user queries with a contextual context and generate responses in a seamless, integrated way. It includes memory management across so the assistant can remember past queries, retrieved documents, or instructions. Assistants can be configured with specialized instructions, behaviors, parameters other than custom tools makes this API far more powerful for developing RAG.

This subsection provides a step-by-step guide and code snippets to the OpenAI's **File Search** within the Assistant as illustrated

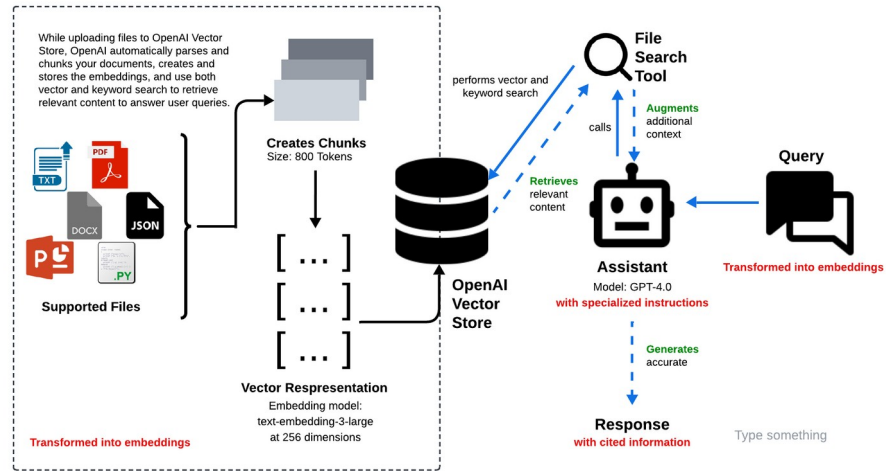


Fig. 2: Open AI's Assistant API

The diagram shows how after the domain data ingestion of supported files (such as PDFs, DOCX, JSON, etc.), the preprocessing[2] and vectorization[3] is handled by Assistant API. These are stored in OpenAI **Vector** which the File Search can query retrieve relevant content. The assistant then augments the context and accurate responses based on specialized instructions and the retrieved tion. This integrated process is covered in detailed steps

1. Environment Setup and API

Setting up your environment and configuring access to OpenAI's API is foundational

- Create an OpenAI
- Once logged in, navigate to the OpenAI API dashboard. Generate a Project API
- Depending on your usage and plan, OpenAI may require you to set billing information. Navigate to the Billing section in the dashboard add your payment details. Refer to Appendix[7] for cost

Store your API key A **.env** file is used to securely store ment variables, such as your OpenAI API

- Set Up a New Project Folder and Environment:** create a new folder for your project. Ensure that a virtual is already set up in this folder, as described in

- (b) **Create a .env File:** Inside your new folder, make a file called `.env`. This file will store your OpenAI API key.
- (c) **Add Your API Key:** Open the `.env` file and paste your OpenAI key in this

```
OPENAI_API_KEY =
```

Be sure to replace with your API key

- (d) **Save the .env File:** After adding your key, save the `.env` file in the same folder where you'll keep your Python code.
- (e) **Install Necessary Python Packages:** To make everything work, you need two packages: `openai` and `python-dotenv`. Open a terminal (or Command Prompt) and run this command to install them:

```
pip install python-dotenv openai
```

If you need specific versions of these tools used for the code in this repository, you can specify them like

```
pip install python-dotenv==1.0.1 openai
```

- (f) **Create the Main Python File:** In the same folder, create a file called `main.py`. The code snippets attached in this entire section [4.2.1] should be implemented within this file. To interact with the OpenAI API and load environment variables, you need to import the necessary libraries. The `dotenv` library is used to load environment variables from the `.env` file.

Code Example: Import

```
import os
import openai
import time
from dotenv import load_dotenv
```

Next, you need to load the environment variables from the `.env` file and set up the OpenAI API client. This is important for authenticating your requests to the OpenAI service and setting up the connection to interact with OpenAI's Assistant.

Code Example: Set OpenAI API Key and

```
# Load environment variable from .env file
load_dotenv()

# Check if OPENAI_API_KEY is set
openai_api_key = os.getenv("OPENAI_API_KEY")
if not openai_api_key:
    raise EnvironmentError("Please set the OPENAI_API_KEY in the .env file.")
```

```
# Set OpenAI key and model
openai.api_key = openai_api_key
client = openai.OpenAI(api_key=openai.api_key)
model_name = "gpt-4o" # Another model from GPT series
```

2. Understanding the Problem Domain and Data

To develop an effective solution for managing and retrieving information, it is important to understand the problem domain and identify the specific requirements and *what you need to provide*. For a deeper insight into the challenges of handling PDFs, refer to [1]. Given that this section focuses on working with PDFs, it is important to emphasize the importance of having relevant and clean data within these documents.

Organize the Knowledge Base Files After selecting the PDF(s) for your external knowledge base, create a folder named **Upload** in the current directory, and place all the selected PDFs inside this folder.

Common Mistakes and Best Practices

Mistake: Irrelevant or inconsistent data Poor-structured or inconsistent data in PDFs downgrades the quality of embeddings generated for Language Models (LLMs), hindering them to understand and retrieve the content more effectively.

Best Practice: Ensure data consistency and relevance PDFs uploaded to the vector store are clean, consistent, and highly relevant to the problem at hand.

Best Practice: Use descriptive file names and metadata Using descriptive file names and adding relevant metadata can significantly aid in the maintenance and retrieval of files. Files should be named in a way that reflects their content or relevance to the system.

The following Python code defines a function to upload multiple PDFs from a specified directory to OpenAI vector store, which is a common structure used for storing and querying high-dimensional vector data in machine learning and AI applications. It ensures the directory and files are valid before proceeding with the upload and collects and returns the loaded files.

NOTE: The function is called to run only when a new vector store is created, meaning it won't upload files to an existing vector store. You may modify the logic as needed to suit your requirements. Also please be aware that the files are stored on OpenAI servers such as Azure OpenAI.

OpenAI's user privacy and data policies. They state they do not use customer data to train their models unless explicitly permitted the user. For more details refer: <https://openai.com/policies/privacy>. Additionally, the data stored can be deleted easily when necessary either code: <https://platform.openai.com/docs/api-reference/files/delete> or from user interface by clicking the delete button <https://platform.openai.com/storage/>

Code Example: Upload PDF(s) to the OpenAI Vector

```
def upload_pdfs_to_vector_store(directory_path, vector_store_id):
    try:
        if not os.path.exists(directory_path):
            raise FileNotFoundError(f"Directory '{directory_path}' does not exist.")
        if not os.listdir(directory_path):
            raise ValueError(f"Directory '{directory_path}' is empty. No files to upload.")

        file_ids = {}
        # Get all PDF file paths from the directory
        file_paths = [os.path.join(directory_path, file)
                       for file in os.listdir(directory_path)
                       if file.endswith(".pdf")]

        # Check if there are any PDFs to upload
        if not file_paths:
            raise ValueError(f"No PDF files found in directory '{directory_path}'.")

        # Iterate through each file and upload to vector store
        for file_path in file_paths:
            file_name = os.path.basename(file_path)

            # Upload the new file
            with open(file_path, "rb") as file:
                uploaded_file = client.beta.files.upload(
                    vector_store_id=vector_store_id,
                    file=file
                )
            print(f"File '{file_name}' with ID {uploaded_file.id} uploaded successfully.")
            file_ids[file_name] = uploaded_file.id

        print(f"{len(file_ids)} files have been successfully uploaded to vector store with ID {vector_store_id}")
```

```

        retur file_id

    except Exceptio as e:
        print ( f "      uploadin file to vecto store {e}
              " )
        retur Non

```

3. Creating and Managing Vector Stores in

OpenAI Vector stores are used to store files for use by the *file search* tool Assistant API. This step involves initializing a vector store for storing embeddings of documents and retrieving them when

Code Example: Initialize a Vector Store for

```

# Get /      Vecto Stor
def get_or_create_vector_store ( client      vector_store_name )
    i no vector_store_name
        rais ValueError ( "      'vector_store_name' i
              no set. Pleas provid a vali vecto stor
              name ." )

    try

        # Lis al existin vecto store
        vector_store = client . beta . vector_stores . list

        # Chec i the vecto stor wit the give name
        alread exist

        fo vector_stor i vector_stores .
            i vector_store .      == vector_store_name
                print ( f "      Stor '{ vector_store_name }
                      alread exist wit ID { vector_stor
                      .
                retur vector_stor

        # Creat a ne vecto stor i i doesn't exis
        vector_stor = client . beta . vector_stores . create
            name =
        print ( f "      vecto stor '{ vector_store_name }
            create wit ID { vector_store . id }

        # Uploa PDF to the newl create vecto stor (
            assumin 'Upload i the director containin
            PDFs
        upload_pdfs_to_vector_store      vector_store
            id 'Upload
        retur vector_stor

    except Exceptio as e:

```

```
print ( f "      creatin o retrievin vecto store
      { e }" )
return Non
```

Common Mistakes and Best

Mistake Ignoring context and query augmentation strategies Relying solely on the vector embeddings without query-specific context or augmentation can lead to re sponses.

Best Practice: Augment Queries with Contextual tio Incorporate contextua information when ing queries to improve quality Using techniques like evance feedback or pseudo-relevance feedback can help refine results. [[6]]

Best Handle Naming Conflicts Whe creating vector stores, consider adding a timestamp or unique tifier to the vector store name to avoid naming conflicts and make easier to manage multiple vector

Best Chunking By default OpenAI uses *max chunk size tokens 800* and *chunk overlap tokens 400* to chunk the file(s) for Vector Stores. Properly sized chunks ensure each chunk contains a coherent and contextually meaningful piece informationI chunks are too they may contain content, conversely, if chunks are too small, they may lack context to be useful. Configure the variables accordingly the

Once the functions to upload PDF file(s) and creating a vector store defined you can call it to create **Knowledge Base** for your project by viding **vector store name** and store in a *vector store* object as shown

Code Example: Creating Vector Store Object

```
vector_store_name = " " # Ensur thi i set to a vali
name
vector_store = get_or_create_vector_store ( client
vector_store_name
```

4. Creating Assistant with Specialized

After setting up the vector store, the next step is to create an AI using the OpenAI This assistant be configured with instructions and tools to perform RAG tasks Set the **assistant name**, **description** and **instructions** properties accordingly. Refer to

best practices, if needed you can also play with the **temperature** and **p** values as per the project needs for random or deterministic

Code Example: Create and Configure Assistant

```
# Get / Create Assistant
def get_or_create_assistant(model_name,
                             vector_store_id):

    assistant_name = "" # Ensure this is set to a valid
    name
    description = "" # Ensure Purpose of Assistant is set
    here
    instruction = "" # Ensure Specialize Instruction
    for Assistant and Conversation Structure is set
    here

    try
        assistant = client.beta.assistants.list
        for assistant in assistants:
            if assistant.name == assistant_name:
                print("Assistant already exist with ID: " + assistant.id)
                return assistant

        assistant = client.beta.assistants.create(
            model=model_name,
            name=assistant_name,
            description=description,
            instructions=instructions,
            tools=[{"type": "file_search"}],
            tool_resources={"file_search": {"vector_store_ids": [vector_store_id]}},
            temperature=0.7 # Temperature for sampling
            top_p=0.9 # Nucleus sampling parameter
        )
        print("A Assistant create with ID: " + assistant.id)
        return assistant

    except Exception as e:
        print(f"Error creating or retrieving assistant: {e}")
        return None

assistant = get_or_create_assistant(model_name,
                                     vector_store_id)
```

Common Mistakes and Best

Best **Ask the** **to adopt a Persona** Provide specialized context-rich *instructions* that guide the assistant how to handle queries, what tone to use (e.g., formal, friendly), which domains to This ensures the assistant more accurate and contextually appropriate

Best Practice: Use inner monologue or Conversation The idea of inner monologue as a *instructions* is instruct the model to put parts of the output into a structured mat. This help understand the reasoning process that a model to arrive at a final

Best Practice: Fine-Tune Model Parameters Based on Adjust parameters such as *temperature* (controls ness) and *top p* (controls diversity) based on the application. This can impact the coherence and creativity of the assistant's puts. For a customer support assistant, a lower temperature may preferable for consistent responses, while a more creative might benefit from a higher

Best Practice: Classifying Queries into For in which lots of independent sets of instructions are needed to different cases, it can be beneficial to first classify the type of and to use that classification to determine which instructions needed.

5. Creating Conversation

Creating a thread, initializes a context-aware conversation session where AI assistant can interact with the retrieve relevant information the vector store, and generate responses based on that context. *tool resources* can be attached to the Assistant API threads that are available to the assistant's tools in this

This capability is essentially important when there is a need to use the AI assistant with different tools for different They can be cally managed to suit the requirements for topic-specific threads, reusing same Assistant across different contexts or overwriting assistant tools for specific

Code Example: Initialize a thread for

```
# Create thread
thread_conversation = {
    "tool_resources": {
        "file_search": {
            "vector_store_ids": [vector_store_id]
```

```

    }
}

message_thread = client.beta.threads.create
thread_conversation

```

6. Initiating a

A **Run** represents an execution on a thread. This step involves sending input to the assistant, which then processes it using the associated retrievers and returns a response that could dynamically fetched citations or data from relevant

This following code allows a user to ask questions to an assistant in a loop. It sends the user's input to the assistant, waits for the assistant to think and respond, and then displays the response word by word. The process repeats until the user types "exit" to

Code Example: Interact with the

```

# Interact with assistant
while True:
    user_input = input("Enter your question (or type 'exit' to quit): ")

    if user_input.lower() == 'exit':
        print("Ending the conversation. Goodbye!")
        break

    # Add a message to the thread with the new user input
    message_conversation = {
        "role": "user",
        "content": [
            {
                "type": "text",
                "text": user_input
            }
        ]
    }

    message_response = client.beta.threads.messages.create(
        thread_id=message_thread_id,
        message_conversation=message_conversation
    )

    run = client.beta.threads.runs.create(
        thread_id=message_thread_id,
        assistant_id=assistant_id,
        message_ids=[message_response.id]
    )

```

```

)

response_text = ""
citation = []
processed_message_id = set()

while True:
    run_status = client.beta.threads.runs.retrieve(
        thread_id=message_thread_id,
        run_id=run_id
    )
    if run_status.status == 'completed':
        break
    elif run_status.status == 'failed':
        raise Exception(f"Run failed with status {run_status.status}")
    time.sleep(1)

while True:
    response_messages = client.beta.threads.messages.list(
        thread_id=message_thread_id
    )

    new_messages = [msg for msg in response_messages.data if msg.id not in processed_message_ids]

    for message in new_messages:
        if message.role == "assistant" and message.content:
            message_content = message.content[0].text
            annotation = message_content.annotations

            for index, annotation in enumerate(annotation):
                message_content.value = message_content.value.replace(
                    annotation.text, f"[{index}] "
                )
                if file_citation := getattr(annotation, "file_citation", None):
                    cited_file = client.files.retrieve(file_citation.file_id)
                    citations.append(f"[{index}] {cited_file.filename}")

            word = message_content.value.split()
            for word in words:
                print(word, end=" ", flush=True)
            time.sleep(1)

            processed_message_ids.add(message.id)

    if any(msg.role == "assistant" and msg.content for msg in new_messages):

```

```

        break

    time.sleep(1)

    citations = []
    print("\nSources: ", ".join(
    print("\n

```

The flexibility of configuring the assistant and thread or assistant tool using OpenAI's API makes this approach highly versatile for various use cases. By following the steps outlined in this section and adhering to the provided best practices, developers can effectively build a powerful RAG application.

4.2.2 Using Open-Source LLM (Llama) We will utilize an open-source framework that implements the Llama model to leverage Llama-based question generation capabilities within our application. By employing this framework, we can process user input and generate contextually relevant questions directly through the terminal, offering an efficient and scalable solution for language processing tasks in a local environment. This approach will enable seamless question generation without relying on external APIs, ensuring both privacy and computational efficiency.

1. Install the following Python packages

```

pip install pymupdf langchain - faiss -
pip install ollama sentence - sentencepiece
pip install langchain -

```

2. Converting PDFs to Text

Save this script as pdf_to_text.py. This script converts PDF files in a folder into text. You have to create one folder at the same level as the script. The folder name should be pdf. You have to keep your PDF files in the pdf folder for the further process. Check the GitHub Link of the script.

Code Example:

```

import os
import fitz  # PyMuPDF for reading PDF

def convert_pdfs_to_text(pdf_folder, text_folder):
    if not os.path.exists(text_folder):
        os.makedirs(text_folder)

    for file_name in os.listdir(pdf_folder):
        if file_name.endswith(".pdf"):
            file_path = os.path.join(pdf_folder, file_name)
            text_file_path = os.path.join(text_folder, file_name.replace(".pdf", ".txt"))
            with open(text_file_path, "w") as text_file:
                text = fitz.open(file_path).get_text()
                text_file.write(text)

```

```

text_file_nam = os . path . splittext
[0] + ".txt"
text_file_pat = os . path . join
text_file_name

with fitz . open as doc :
    text = ""
    for page i in doc :
        text += page . get_text ()

with open "w" encoding = "utf
" ) as text_file
    text_file . write
print ( f " { file_name } to {
text_file_name }"

if __name__ == "__main__"
    pdf_folde = "Data
    text_folde = "DataTxt
    convert_pdfs_to_text text_folder

```

Functionality: Converts all PDFs in the Data folder to text files and them in the DataTxt

3. Creating the FAISS

Save this script as txt to This script generates a FAISS from the text files in the DataTxt folder. Check the GitHub link for the

Code Example:

```

import os
from langchain_huggingfac import HuggingFaceEmbedding
from langchain_community import FAIS

def load_text_files
    texts = []
    for file_name i in os . listdir
        if file_name . endswith ( ".txt"
            file_pat = os . path . join
            file_name
            with open "r" encoding = "utf-8" as
            file
            texts . append ( file . read
        return texts

def create_faiss_index index_path
    embedding_model = 'sentence-transformers/all-MiniLM-v2'):
    texts = load_text_files

```

```

embedding = HuggingFaceEmbeddings ( model_name
embedding_model
vector_store = FAISS . from_texts ( texts embeddings

vector_store . save_local
print ( f " index saved to { index_path }

i _name_ == " _main_
text_folder = " DataTxt
index_path = " DataIndex
create_faiss_index index_path

```

Functionality: Creates a FAISS index and saves it in the DataIndex/ Here, sentence-transformers/all-MiniLM-L6-v2 is a compact, fast former that generates sentence embeddings for tasks like search and text It's efficient and for accurate retrieval in applications like FAISS

4. Setting Up Ollama and Llama

Before implementing the last script that handles user queries and responses using a Large Language Model (LLM), we need to select an source LLM. By using Ollama as the model runner, we can easily Llama 3.1 into our

(a) Step 1: Download Ollama

To get started with Ollama, follow these

- i Visit the official Ollama
- ii Choose the version suitable for your operating system macOS, or
- iii Download and install the appropriate installer from the

(b) Step 2: Install Llama 3.1 Using

After installing you can download and Llama 3.1 running the following command in your PowerShell or CMD

```
oLlam pul Llama3
```

(c) Step 3: Running Llama

Once the Llama 3.1 model is installed, you can start using it by a command similar to

```
oLlam ru Llama3
```

This command runs the Llama 3.1 model, allowing you to ask directly in the terminal and interact with the model in real

(d) **Step 4: Test Ollama in VS**

create a .bat file to avoid typing the full path each time. Open add

```
@echo o
" C :\ path \ to \ Ollama \ oLlama . exe%*
```

Replace the path with your own. Save the file as oLlama.bat directly your project

In the VS Code terminal, run the .bat file with the

```
.\ oLlama .      ru   Llama3
```

5. **Implementing RAG-Based Question**

Save this script as This script retrieves relevant documents FAISS and generates questions based on the retrieved context using and Llama 3.1. Check the GitHub

Code Example:

```
import os
from langchain_community . import FAIS
from langchain_huggingfac import HuggingFaceEmbedding
from langchain . import PromptTemplat
from langchain . import RetrievalQ
from langchain_community . import OLLam

# Loa FAIS inde
def load_faiss_index ( index_path , embedding_model )
    # Loa the FAIS inde usin the same embeddin model
    embedding = HuggingFaceEmbeddings ( model_name
        embedding_model
    vector_stor = FAISS . load_local ( index_path embedding
        , allow_dangerous_deserialization = True
    retur vector_stor

# Creat the RA system usin FAIS and OLLam ( Llam 3.1)
def create_rag_system ( embedding_model =
    sentence - transformers / all - MiniLM -L6 - v2 model_name ="
    Llama3 .1
    # Loa the FAIS inde
    vector_stor = load_faiss_index
        embedding_model

    # Initializ the OLLam model ( Llama3
    ll = Ollama ( model =

    # Creat a mor detaile promp template
    prompt_templat = ""
```

```

You are an expert assistant with access to the
following context extracted from documents. You
join to answer the user question as accurately
as possible using the context below

Context
{ context

Give this information please provide a comprehensive
and relevant answer to the following question
Question { question

If the context does not contain enough information
clearly state that the information is not
available in the context provided
If possible provide a step-by-step explanation and
highlight key details
"""

# Create a template for formatting the input for the
model
prompt = PromptTemplate
    input_variables=[ "context" "question"
    template =
)

# Create a RetrievalQA chain that combines the vector
store with the model
qa_chain = RetrievalQA.from_chain_type
    llm =
    chain_type = "stuff"
    retriever = vector_store.as_retriever()
    chain_type_kwargs={ "prompt" prompt
)

return qa_chain

# Function to run the RAG system with a user question
def get_answer(question, qa_chain):
    answer = qa_chain.run(question)
    return answer

if __name__ == "__main__":
    # Path to the FAISS index directory
    index_path = "DataIndex"

    # Initialize the RAG system
    rag_system = create_rag_system(index_path)

    # Get user input and generate the answer

```

```

while True
    user_question = input ( "      you question ( o type
                        'exit to quit ) " )
    if user_question . lower == " exit
        print ( "      the RA system . ")
        break
    answer = get_answer ( user_question , rag_system )
    print ( f "      { answer } " )

```

Functionality: The script takes a user query from the terminal. It retrieves relevant documents using FAISS. Then it generates an answer using the retrieved context with OLLama and Llama

Common Mistakes and Best Practices

- Incompatible embedding models:** The FAISS index is typically created using a specific embedding model. If a different embedding model is used during querying, a version mismatch occurs, leading to errors. Always ensure that the same embedding model is used both during indexing and querying.
- Model version issues:** Using an incorrect or unsupported version (e.g., referencing a non-existent version like Llama 4.5) can lead to failures during the model loading process. Always verify that the model version you are using is supported.
- Overly general prompts:** Prompts that are too broad or vague can result in irrelevant responses from the LLM. Craft precise and targeted prompts to ensure more accurate and useful answers.
- Ignoring context:** Language models can generate incorrect or hallucinated responses when the retrieved documents lack the necessary information, leading to gaps in the generated text. Always ensure sufficient context is provided in the prompts.
- Memory management:** Extended use of FAISS and OLLama in loops without proper memory management can result in memory leaks, gradually consuming system resources. Monitor memory usage and free up resources after each loop to avoid system slowdowns.
- Model re-initialization:** Reloading or re-initializing models unnecessarily can slow down your system. Reuse initialized models whenever possible to improve system efficiency and reduce latency.

OLLama (Llama 3.1) is a language model that runs entirely on the user's hardware, ensuring data privacy and faster response times on the system's hardware. The accuracy of its outputs depends on the

ity of its training and it can be further improved by fine-tuning domain-specific Fine-tuning involves retraining the wit specialized datasets, allowing it to internalize specific organizational edge for more precise and relevant responses. This process keeps the updated and tailored to the user’s needs while maintaining

5 Preliminary Evaluation of the

5.1 Feedback Process Overview

This experience report underwent an informal evaluation process aimed at ering feedback for the section: **Using OpenAI’s Assistant API : GPT ries.** Although the feedback session was not formally it provided valuable insights that helped validate the ideas presented in this tion and refine the guide based on it. The feedback gathered from demonstrates that the workshop was A majority attendees able to follow the provided guide and successfully implemented their RAG els by the end of the

5.2 Participant

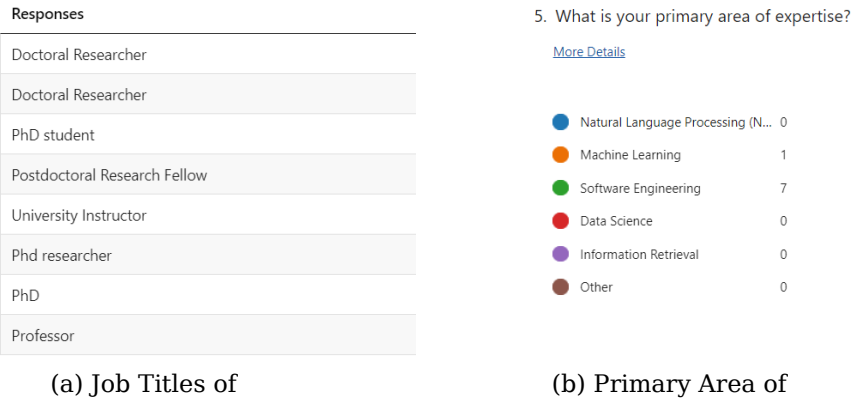


Fig. 3: Demographic Information from

We collected feedback from a but diverse group of participants during workshop. A total of 8 individuals completed a demographics form, which vided us with an understanding of the participants’ backgrounds and expertiseThe group consisted of individuals with varying levels of

in machine learning, natural language processing (NLP), and using tools for retrieval Augmented Generation (RAG). The participants although had with Python language and OpenAI

5.3 Key Feedback Points

During the participants shared their thoughts on how much their derstanding RAG systems improve after the which aspect the workshop did they find most challenges they faced and what tions or comments they want to provide for future improvement.Below are key points highlighted by the

8. Before attending the workshop, how familiar were you with Retrieval-Augmented Generation (RAG) systems?

[More Details](#)



Fig. 4: Participants' Familiarity with RAG

Prior to attending the workshop, the majority of participants reported a sonable level of familiarity with RAG systems.This indicated that the had a foundational understanding of the concepts presented, allowing for more depth discussions during the workshop. After the workshop, there was a improvement in participants' understanding of RAG

9. How much did your understanding of RAG systems improve after the workshop?

[More Details](#)

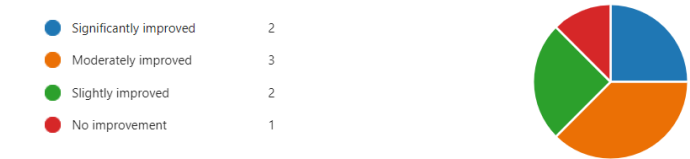


Fig. 5: Participants' Improvement in Understanding RAG

The majority of participants highlighted the practical coding exercises as most valuable aspect of the workshop, which helped them better understand

10. Which aspect of the workshop did you find most valuable?

[More Details](#)

● Theoretical explanations of RAG	0
● Practical coding exercises	5
● Discussions with peers and instr...	3
● Q&A sessions	0
● Other	0



Fig. 6: Most Valuable Aspects of the

implementation RAG. Additionally, several participants found the discussions with peers and instructors as a key

5.4 Incorporating Feedback to Improve the

The evaluation also revealed opportunities for improvement of guide, in enhancing the clarity instructions and streamlining the process. The most common issues raised were most of them to copying from PDF file generating errors as numbers. In addition that we implemented error handling to throw errors to the user the code snippets provided to seamlessly run the

2	anonymous	Well structured of instructions but copy code should be with terminal command
3	anonymous	Copy-pasting code from the pdf files.
4	anonymous	Some code problems...
5	anonymous	it is very easy to understand
6	anonymous	Havent face any challenges
7	anonymous	Editor and environment issues not relating to the guide.

Fig. 7: Feedback on challenges faced during the implementation of the

Several participants also provided suggestions for future. On notable suggestion was to include a warning regarding sensitive data in vector store. The detailed comment are shown in

In the evaluation process proved valuable in validating the proach outlined in the guide. By testing it in a hands on workshop and in an open discussion sessions of how the developed RAG models

12. Any other comments or suggestions for future improvement?

3 Responses

ID ↑	Name	Responses
1	anonymous	My last remark is about the vectoreStore, I just found out it is stored in OpenAI's server, so there should be a warning to not upload any sensitive documents. And also, I want to know , in the guide: where it is stored, and how to delete it.

Fig. 8: Comments and suggestions for improving the

trustworthiness in specific we were able to collect feed-back and directly address areas of difficulty faced by practitioners. The driven improvements have not only made the guide more user friendly but demonstrated the importance of continuous iteration based on real world

6 Discussio

Practitioners in fields like healthcare, legal analysis, and customer support, struggle with static models that rely on outdated or limited RA models provide practical solutions with pulling in real time data from sources. The ability to explain and trace how RAG models reach their also builds trust where accountability and decision making based on real is

In this paper, we developed a RAG guide that we tested in a workshop where participants set up and deployed RAG systems following the mentionedThis contribution is as it helps practitioners RAG models to address real world challenges with dynamic data and accuracy. The guide provides users clear, actionable steps to integrate RAG their workflows, contributing to the growing toolkit of AI driven

With that, RAG also opens new research avenues that can shape the of AI and NLP technologies. As these models and tools improve, there are potential areas for growth, such as finding better ways to search for adapting to new data and handling more than just text images or Recent advancements in tools and technologies have accelerated the development and deployment of RAG models. As RAG continue to evolve, several emerging trends are shaping the future of this

1. **Haystack:** An open-source framework that integrates dense and sparse trieva methods with large-scale language Haystack supports time search applications and can be used to develop RAG models that form tasks such as document retrieval, question answering, and tion
2. **Elasticsearch with Vector** Enhanced support for dense search capabilities, allowing RAG models to perform more sophisticated trieva tasks Elasticsearch's integration with frameworks like Faiss

hybrid retrieval systems that combine the strengths of both dense and search methods, optimizing retrieval speed and accuracy for large

3. **Integration with Knowledge Graphs:** Researchers are exploring ways integrate RAG models with structured knowledge bases such as graphs. This integration aims to improve the factual accuracy and capabilities of the models, making them more reliable for knowledge-tasks[8]
4. **Adaptive Learning and Fine-Tuning:** There is a interest in adaptive learning techniques that allow RAG models to uously fine-tune themselves based on new data and user feedback. This proach aims to keep models up-to-date and relevant in rapidly information
5. **Cross-Lingual and Multimodal Capabilities:** Future RAG models expected to expand their capabilities across different languages and ities Incorporating cross- retrieval and data can make RAG models more versatile and applicable to a wider range global and multimedia

Future research will likely focus on enhancing their adaptability, cross-capabilities and integration with diverse data sources to address complex information

7 Conclusion

The development Retrieval-Augmented Generation (RAG) systems offers new way to improve large language models by grounding their outputs in time, relevant information. This paper covers the main steps for building systems that use PDF documents as the data source. With clear examples code it connects theory with practice and highlights challenges handling complex PDFs and extracting useful text. It also looks at the available, with examples of using proprietary APIs like OpenAI's GPT and, an alternative, open-source models like Llama 3.1, helping developers choose best tools for their

By following the recommendations in this guide, developers can avoid mon mistakes and ensure their RAG systems retrieve relevant information generate fact-based As technology advances in learning, multi-modal capabilities, and retrieval methods, RAG systems will a key role in industries like legal research and documentation This guide offers a solid foundation for optimizing RAG systems extending the potential of generative AI in practical

References

1. Avi Arampatzis, Georgios Peikos, and Symeon Pseudo relevance back *Information Journal*, 24(4-5):269-297, May
2. Md Chowdhury, John Smith, Rajesh Kumar, and Sang-Woo Cross-lingual multimodal retrieval-augmented generation *IEEE Transactions on media*, 27(2):789-802,
3. Elasticsearch Integrating dense vector search in Elastic Blog,
4. Haystack The haystack framework for neural Haystack Project tation,
5. Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Naman Goyal, and Sebastian Riedel. Retrieval-augmented generation for intensiv nlp In *in Information Systems (NeurIPS 2020)*,
6. Hang Li, Ahmed Mourad, Shengyao Zhuang, Bevan Koopman, and Guido Pseudo relevance feedback with deep language models and dense Successes and *ACM Transactions on Information* 41(3):1- April 2023
7. Percy Wen-tau Douwe and Sebastian Best practices training large language models: Lessons from the field. *IEEE Transactions on Networks and Learning Systems*, 34(9):2115-2130,
8. Chenyan Xiong, Zhuyun Dai, Jamie Callan, and Jie Knowledge-enhanced guage models for information retrieval and *IEEE Transactions on edge and Data Engineering*, 36(5):1234-1247,

Appendix

1. Tampere University, "Cost Estimation for RAG Application Using Zenodo, Sep. 2024. doi: